ZK Proofs for TSP — Constructions Considered

Flat baseline (Ch. 4) and the hierarchical / recursive family (Ch. 5)

Working report for supervisor review — not thesis prose. Author: [name] · Date: [date]

Purpose. This is a compact map of every construction we considered, so you can sanity-check the design choices and the soundness/privacy claims before we commit the chapter. It is deliberately terse: each construction is given as *problem → solution → soundness → privacy → cost.* Magnitudes are from our own measurements; the thesis defers the full numbers to Ch. 6.

Notation. N = nodes; the prover knows a Hamiltonian cycle (a closed tour visiting each node once). K = number of segments; $M = N/K$ = segment size. T = public cost threshold. root = Poseidon2 Merkle root committing the $N \times N$ cost matrix. "Hiding" always means hiding of the *partition* (which nodes are grouped, their order, per-segment costs).

1. The starting point: the flat baseline (Chapter 4)

What is proved. The prover convinces a verifier that it knows a Hamiltonian cycle on N nodes whose total cost is $\leq T$ against the committed cost matrix, **revealing nothing else** (not the route, not the matrix entries).

Matrix commitment. The $N \times N$ matrix is flattened and committed as a Poseidon2 Merkle root. Two regimes:
- **flat-full**: the matrix is public (verifier re-hashes its own copy; integrity only).
- **flat-Merkle**: only root is public (privacy + integrity). **This is the regime we carry.**

One monolithic circuit, three constraint groups.
- **Permutation**: the N visited nodes are exactly $\{0, \ldots, N-1\}$, each once.
- **Edge cost**: every edge cost is bound to the matrix, via a Merkle proof whose leaf index is pinned to the exact (from, to) cell (blocks substituting a cheaper cell).
- **Threshold**: the sum of the N edge costs is $\leq T$.

Five ways to check the permutation (the mechanism study). pairwise compare · sort · inverse-permutation · presence/lookup · **grand product**. We carry two as the workhorses:
- **sort**: $\text{sort}(\text{cycle}) = [0..N-1]$. Exact, unconditional, simple.
- **grand product**: $\prod_j (X + \text{node}_j) = \prod_{j=0}^{N-1} (X + j)$ at a challenge X. Compact (one field element), but **probabilistic** (Schwartz–Zippel) and needs a challenge.

Witness-time inversion (a measured curiosity, carried into Ch. 5). The grand product compiles to slightly *more* gates than the sort, yet solves its witness *faster* (the sort's data-dependent memory is expensive to solve). Same proof size. "Gate count and witness cost are different currencies."

The wall. The flat circuit does not scale: gates, witness, and peak memory all grow with N and all land on one machine. Past roughly $N \approx 500$–1000 a single prover runs out of memory. This is the problem Chapter 5 attacks.

Baseline. `flat-sort` is the reference: public surface is just $\{\text{root}, T\}$, so it leaks nothing about the route — **structural** hiding (made precise in the next section). Everything in Ch. 5 is measured against it.

2. Relation to the proof system (UltraHonk)

Every variant runs on the **same** zk-SNARK backend (UltraHonk). To keep the soundness/privacy claims honest we separate two layers; every soundness/privacy entry below lives in this split.

- **Backend (UltraHonk) — fixed, identical across all variants.** Per proof it gives **knowledge soundness** (a valid proof $\Longrightarrow$ the prover knows a witness satisfying the circuit), **zero-knowledge** (the proof hides the witness, and *only* the witness), and **succinctness** (≈14.6 KB proof, fast verify). It rests on its own assumptions (DLOG-class commitments + the ROM for its internal Fiat–Shamir). We use it as a black box and do **not** re-prove it.

- **Our layer — the circuit + the public/private split.** We choose what the circuit proves, the in-circuit cryptographic sub-arguments (Merkle, the grand-product chain, the commitment fold, the recursive verifier), and — decisively — *which values are public vs witness.*

Division of labor:

- **Soundness** = UltraHonk KS (witness-for-circuit) *composed with* our encoding (circuit satisfied $\Longrightarrow$ a real sub-threshold cycle exists) + the in-circuit arguments. By reduction, not from scratch: $\varepsilon \leq \varepsilon_{\text{SNARK}} + \varepsilon_{\text{Merkle}} + \varepsilon_{\text{SZ}} + \varepsilon_{\text{FS}}(+\varepsilon_{\text{bind}} + \varepsilon_{\text{rec}})$. (Note: UltraHonk's internal Fiat–Shamir and our in-circuit $X = H(c)$ are *distinct* ROM uses at different layers.)
- **Privacy is not zero-knowledge.** ZK hides only the *witness* it says nothing about the public inputs, which *are* the statement. A leak is therefore never a ZK failure — it is that we placed the partition in the public inputs. ZK is uniform across all variants and is **not** a discriminator between them.
- **The unifying rule.** UltraHonk's ZK hides the witness, full stop. Every privacy move is about getting the partition *into* the witness (flat, recursion $\longrightarrow$ structural hiding, free under the same ZK) or wrapping it when it must stay public (committed $\longrightarrow$ needs Poseidon-as-random-oracle + secret r; that one extra assumption *is* the gap to flat). Plaintext left on the surface (plain) $\longrightarrow$ ZK cannot help $\longrightarrow$ leak.
- **Binding, two senses.** The backend binds *proof* $\leftrightarrow$ *its public inputs* (what makes the cross-check meaningful); our hash binds *commitment* $\leftrightarrow$ *values* (Poseidon2 collision resistance).
- **Metrics measure cost, not guarantees.** `proof_bytes` and `verify_s` are uniform (succinctness + a fixed ZK overhead) — you cannot read privacy or soundness off them. `circuit_size` / `prove_s` / `peak_mb` price the constraints that *buy* the guarantees.

3. Two findings that organize Chapter 5

Finding 1 — the dualism (a negative result). Decomposing the *problem* helps an optimizer (it adds a constraint, shrinking the search). Decomposing the *proof* does the opposite (it weakens a constraint, which must be restored at full price). The prover has no search to prune, so:

- **Total work is conserved.** Hierarchical is never cheaper than flat — slightly *more* (a few % at large N, more at small N / large K). **There is no crossover.**
- What decomposition *does* buy: the same work made **portable** (fits per machine, clears the wall) and **parallel** (segments proved independently).

Finding 2 — the stitching tax. Recombining K independent proofs into one trusted statement is **one cost with three faces**, all from a **shared public surface** the independent proofs need in order to relate to each other:

- **leak** — shared values are public;
- $\boldsymbol{O(K)}$ **verifier** — verifier checks $K + 1$ proofs + reconciles them;
- **bookkeeping** — the reconciliation is trusted code outside the circuits.

How hard the tax bites is set by **two decisions**, and the whole family is one choice of each:

- **WHERE** the stitch is enforced: external script $\rightarrow$ in-circuit.
- **WHAT** the shared values are made of: plaintext $\rightarrow$ commitment $\rightarrow$ witness.

stitch regime (WHERE / WHAT)	sort mechanism	product mechanism
flat, monolithic	`flat-sort` (baseline)	`flat-product`
hierarchical, plaintext, external	`plain-sort`	`plain-product`
hierarchical, commitment, external	`committed-sort`	`committed-product`
recursive, witness, in-circuit	`recursive-sort` (control)	`recursive-product` (shipped)

Each construction below is **one move** from the previous along **one** decision. Privacy follows the WHAT; soundness follows the mechanism (sort vs product) and the WHERE.

4. The hierarchical / recursive constructions

4.1. `plain-sort` — decompose, stitch externally on plaintext

- **Problem.** The flat circuit hits the wall. Cut the cycle into K segments of M nodes.
- **Solution.** Three layers: **(i)** each **segment** proves a valid M-node path (range, sorted node set bound to its real nodes, endpoints pinned, $M-1$ internal edges Merkle-checked, partial cost = sum); **(ii)** a **glue** circuit restores the global checks (coverage by sorting all N published nodes to $[0..N-1]$; K boundary edges; threshold); **(iii)** an **off-circuit cross-check** equates the values shared between glue and segments (same root, node sets, endpoints, costs).
- **Why each layer.** Segment *binds* its summary to its private nodes; glue *enforces* the global properties; cross-check *ties* the separately-proved values together. A single `bb verify` only certifies one proof against its own public inputs — nothing links two proofs without the cross-check (else $K+1$ valid proofs could describe $K+1$ unrelated instances).
- **Soundness.** Exact, unconditional (the coverage sort is an algebraic identity).
- **Privacy. None** — the partition (node sets, endpoints, per-segment costs) is published in the clear. At $N=8, K=2$ the route space collapses from 2520 to ≤ 4.
- **Cost.** Cheapest total gates of the family; but the glue's coverage sort is a **serial** $O(N)$ pass.

4.2. `plain-product` — the fingerprint lever (sort → grand product)

- **Problem.** `plain-sort` publishes M nodes per segment ($O(M)$ surface) and the glue sorts serially.
- **Solution.** Each segment publishes one field element $P_i = \prod_v (X+v)$ instead of its node list. Glue checks $\prod_i P_i = \prod_{j=0}^{N-1}(X+j)$ (coverage via product). Surface drops $O(M) \to O(1)$; the permutation work distributes across segments.
- **The catch — the challenge X.** The product is sound only if X is unpredictable until the cycle is fixed (else the prover root-hunts a fake partition). Fiat–Shamir: $X = H(c)$ where c is a Poseidon2 hash **chain over all N nodes in order**. The chain is split across segments via two anchors per segment ($h_{\text{in}}, h_{\text{out}}$); the glue stitches them (h_{in} of segment $0 = 0$; seam $h_{\text{in}[i+1]} = h_{\text{out}[i]}$; close $h_{\text{out}[K-1]} = c$). Each node must be folded in, or one honest segment lets the prover settle the rest after seeing X.
- **Soundness. Probabilistic**: Schwartz–Zippel ($\leq N/|\mathbb{F}| < 2^{-240}$) + Fiat–Shamir in the random-oracle model + Poseidon2 collision / preimage resistance. (Trade: structural certainty → computational.)
- **Privacy. Still none.** P_i and the anchors are a **confirmation oracle**: enumerate the $C(N,M)$ subsets and the $(M-2)!$ orderings, match against the public values. A search wall that vanishes at small N and is no obstacle to an adversary who already has a candidate.
- **Cost / role.** A few % more total gates than `plain-sort` (the chain), but a lighter, distributable glue + faster witness (neither dominates). **Built now mainly as the substrate recursion needs:** its $O(1)$, M-independent surface is what keeps a later recursive verifier flat in M.

4.3. `committed-sort` / `committed-product` — hide the surface (plaintext → commitment)

- **Problem.** The plain surface leaks because it is public plaintext.
- **Solution.** Each segment publishes one **blinded commitment** $C_i = \text{fold}(r, \text{values})$ (secret random r, Poseidon2). It hands the actual values to the glue as **private witness**; the glue **re-folds** them with r and checks $= C_i$, then runs the same coverage/connection/threshold on the witnessed values. The cross-check now compares **opaque commitments**.
- **Why it works (two commitment properties).**
  - **Hiding** (secret r + Poseidon-as-RO): C_i reveals nothing; a guess cannot be confirmed without r. **This closes the leak** — and unlike `plain-product`, the hiding does **not** depend on N (holds even at $N=8$, and against a guesser).
  - **Binding** (collision resistance): the re-fold forces the glue's witnessed values to be exactly the committed ones (no made-up-arrays attack).
- **Equal-privacy finding.** A commitment hides M nodes no worse than it hides one scalar, so `committed-sort` and `committed-product` reach the **same** partition-privacy. **This proves the sort↔product lever was**

never a privacy mechanism — only cost/distribution. (One caveat: `committed-product` still publishes X, a whole-cycle, $(N-1)!$ confirmation oracle one level *above* the partition; `committed-sort` does not.)

- **Soundness.** Coverage unchanged per mechanism (sort exact / product probabilistic). New: commitment **binding** (collision resistance) — a fresh assumption only for `committed-sort` (which had none); `committed-product` already relied on it.
- **Privacy.** Partition hidden **computationally** — one assumption below flat (rests on Poseidon-RO
  1. secret r). Residual: K (segment count) is still visible. Pedersen is the available upgrade

  to **unconditional** hiding (DLOG, curve ops, binding becomes computational).
- **Cost.** `committed-product` is the cheaper of the two: its re-fold is $O(1)$/segment, while `committed-sort` must re-fold its whole $O(N)$ node list (its commitment cost does not dilute). Either way modest vs the Merkle work. **The other two tax faces ($O(K)$ verifier, bookkeeping) are untouched** — only WHAT moved, not WHERE.

4.4. `recursion` (`recursive-product`) — move the stitch in-circuit (external → witness)

- **Problem.** Committed still leaks K, keeps the $O(K)$ verifier, and the bookkeeping — all because the stitch is still **external**.
- **Solution.** One **outer** circuit **verifies the K segment proofs in-circuit**, takes their published values as **private witness**, and re-runs the glue logic on them. The cross-check is **absorbed** — the outer circuit *is* the stitch.
- **Why the verify-gadget is essential.** Without it, the outer would run the glue on unbacked witness (made-up-arrays attack). The gadget pins each segment's values to a real segment proof. The shared-X / chain checks still needed (honesty from inner proofs, agreement from the outer).
- **Soundness.** Probabilistic coverage (the inner is forced to be `plain-product`, below) + recursive knowledge-soundness + in-circuit Honk verifier. **Heaviest trust base in the family.**
- **Privacy.** Public surface = $\{\text{root}, T\}$, value-for-value = flat. Hiding is **structural** (the partition is *absent*, not blinded) — rests on nothing flat does not. K leaves the surface (it is only a circuit parameter, like N). Hiding is from the *final verifier*; the aggregator who builds the outer sees the segment values, as flat's prover knows its cycle.
- **Why the inner must be `plain-product` (measured).** The outer's cost scales with the inner's public-input count. `plain-product` carries 9, **independent of M** → outer flat in M (measured: $1.474\text{M} \to 1.483\text{M}$ gates as $M : 48 \to 2000$; the $+0.6\%$ is the glue's $O(N)$ identity product, not the verifications). A `sort` inner has an $O(M)$ surface → outer grows with M (gap ≈ 95 gates/node, never reverses). A committed inner would hide redundantly. So `plain-product` is forced — *this is why we built it leaking.*
- **Cost (the aggregation tax).** One in-circuit verification ≈ 704k gates $\approx 78\times$ the entire `plain-product` glue. Outer is K-linear, M-independent. Total $\approx +82\%$ vs flat at $N = 1000$, $K = 2$. The dualism in its purest form. Parallelism survives **halfway**: segments are parallel, but the outer is one **monolithic** heavy proof. **The win:** the verifier checks **one** proof — the $O(K)$ verifier and the bookkeeping both vanish. (Folding / ClientIVC would parallelize the finish; noted as future work, not built.)

5. The frontier (the synthesis)

Privacy ladder (= the WHAT axis). plaintext (exposes) ⟶ commitment (computational) ⟶ witness (structural). The sort↔product lever moves *along* a rung, never *up* it.

Soundness is a spread, not a ladder (it follows mechanism + WHERE, crossing the privacy order): exact coverage (sort) vs probabilistic (product); external stitch = trusted code vs internal = recursive verification. Net: `plain-sort` has the **lightest** base but hides nothing; `recursion` is the **most private** but carries the **heaviest** base. Exact coverage and structural hiding sit at **opposite ends**.

Pick-two triangle. No construction is cheap, parallel, *and* structurally private at once:

	cheap	parallel	structural privacy	gives up
`flat`	yes	no (the wall)	yes	parallelism

	cheap	parallel	structural privacy	gives up
`committed`	yes	yes	no (computational)	structural privacy
`recursion`	no (agg. tax)	partial (monolithic finish)	yes	cost

The **plain** variants are dominated (committed reaches the same cost + parallelism and adds privacy), so the **deployable frontier is three points: flat, committed, recursion**. The plain variants are rungs we climb, and `plain-product` is recursion's substrate.

6. Consolidated comparison

variant	hiding	coverage soundness	public surface	cost / note
`flat-sort`	structural	exact, uncond.	$\{\text{root}, T\}$	baseline; no parallelism
`plain-sort`	none (plaintext)	exact, uncond.	$O(N)$	cheapest total; serial glue
`plain-product`	none (oracle)	prob. (SZ + ROM)	$O(K)$ + leak	recursion substrate
`committed-sort`	computational	exact + new binding	$O(K)$: C_i	heavy $O(N)$ re-fold
`committed-product`	computational	prob. (SZ + ROM)	$O(K)$: C_i, X	cheaper glue (favoured)
`recursion`	structural	prob. + recursion KS	$\{\text{root}, T\}$	$K \times$ 704k; 1 verifier

7. What we would like validated

- **The reframe:** hierarchical decomposition gives **no** algorithmic speedup for TSP-ZK (no crossover); the value is parallelism + per-prover memory. Is the dualism argument sound?
- **Soundness assumptions per variant** (exact vs Schwartz–Zippel + Fiat–Shamir-in-ROM; commitment binding; recursive knowledge-soundness) — are these the right things to assume and cite?
- **The equal-privacy finding** (committed-sort and committed-product reach the same partition-privacy; the lever is cost-only) — agreed?
- **The forced inner** (recursion must use `plain-product` for an M-independent outer) — agreed?
- **Scope honesty:** `plain-product` leaks and is built only as a stepping stone; recursion's structural hiding costs the steepest aggregation tax and a monolithic finish (folding deferred).
- **Anything you would add, cut, or want proved more carefully before we finalize Chapter 5.**